

INFORME FINAL: ANALIZADOR DE COMPLEJIDADES ASISTIDO POR LLMs

Título del Proyecto: Analizador de Complejidades Asistido por LLMs ("EffiCode
Analyzer")

Asignatura: Análisis y Diseño de Algoritmos

Integrantes del Grupo:

Daner Alejandro Salazar Colorado

Jaime Andres Cardona Diaz

Fecha de entrega: 5 de Diciembre de 2025

2. Introducción

El análisis de algoritmos es una piedra angular en las ciencias de la computación, permitiendo cuantificar la eficiencia de una solución en términos de tiempo y espacio. Sin embargo, la deducción manual de la complejidad asintótica (notaciones Big *Escriba aquí la ecuación.* O , Ω y Θ) suele ser propensa a errores humanos, especialmente cuando se trata de estructuras recursivas complejas o anidamientos lógicos profundos.

El presente proyecto, denominado EffiCode Analyzer, tiene como propósito integrar conocimientos avanzados de diseño de algoritmos con tecnologías emergentes de Inteligencia Artificial. El objetivo principal es construir un sistema híbrido que, a partir de un algoritmo escrito en pseudocódigo estándar (estilo Introduction to Algorithms de Cormen et al.), determine automáticamente su complejidad computacional.

La motivación reside en la necesidad de una herramienta educativa y profesional que no solo entregue una respuesta final, sino que desglose el razonamiento matemático detrás del análisis, utilizando tanto técnicas deterministas (análisis estático de código y resolución de recurrencias) como la asistencia de Modelos de Lenguaje de Gran Escala (LLMs) para la interpretación semántica y validación.

2.1 Objetivos del Proyecto

Objetivo General:

Desarrollar un analizador automático de complejidad algorítmica que combine técnicas de análisis simbólico con inteligencia artificial para proporcionar evaluaciones precisas y fundamentadas matemáticamente.

Objetivos Específicos:

Implementar un parser capaz de interpretar pseudocódigo estilo Cormen, generando un Árbol de Sintaxis Abstracta (AST) que represente la estructura del algoritmo.

Desarrollar módulos de análisis diferenciados para algoritmos iterativos y recursivos, aplicando los métodos formales descritos en el libro de referencia.

Integrar un clasificador neuronal entrenado con más de 20,000 algoritmos para asistir en la identificación de patrones de complejidad.

Incorporar un LLM (Google Gemini) como validador experto que proporcione una "segunda opinión" académica sobre los resultados obtenidos.

Diseñar una interfaz de usuario intuitiva que presente los resultados de forma clara, incluyendo pasos de resolución, costos por línea y visualización del árbol de recursión.

2.2 Alcance del Sistema

EffiCode Analyzer soporta el análisis de:

Algoritmos iterativos: bucles *for*, *while*, *repeat...until*, incluyendo anidamientos múltiples.

Algoritmos recursivos: recursión lineal, binaria (divide y vencerás), exponencial (Fibonacci, Hanoi) y patrones especiales como exponenciación rápida.

Casos de análisis: peor caso (Big O), mejor caso (Big Ω), caso promedio (Big Θ) y análisis probabilístico según el Capítulo 5 del libro de referencia.

El sistema implementa los siete métodos de resolución de recurrencias documentados en Cormen: Teorema Maestro, Método del Árbol de Recursión, Sustitución, Akra-Bazzi, Cambio de Variables, Iteración y Funciones Generatrices.

2.3 Estructura del Documento

El presente informe se organiza de la siguiente manera:

La Sección 3 analiza la naturaleza del problema abordado;

La Sección 4 describe el formato de entrada de datos;

La Sección 5 detalla las estrategias algorítmicas empleadas;

La Sección 6 presenta la arquitectura e implementación del sistema;

La Sección 7 explica la integración de LLMs;

La Sección 8 analiza la eficiencia del propio analizador;

La Sección 9 documenta los casos de prueba;

Finalmente, la Sección 10 presenta las conclusiones y recomendaciones para trabajos futuros.

3. Análisis del Problema

3.1 Naturaleza del Problema

El problema central abordado es la automatización del cálculo de la complejidad temporal $T(n)$ de un algoritmo dado. Este problema presenta desafíos de naturaleza matemática, estructural y semántica que lo convierten en un reto no trivial dentro del campo del análisis estático de programas.

3.2 Características y Desafíos

- **Variedad Estructural:** Los algoritmos pueden ser iterativos (bucles `for/while/repeat...until`) o recursivos. Los métodos de análisis para cada uno son fundamentalmente distintos:
 - Los primeros requieren sumatorias $\sum$ y análisis de límites de iteración.
 - Los segundos requieren la resolución de ecuaciones de recurrencia (ej. $T(n) = aT(n/b) + f(n)$).
 - Además, existen casos híbridos donde algoritmos recursivos contienen bucles internos, multiplicando la complejidad del análisis.
- **Ambigüedad del Lenguaje Natural:** Los pseudocódigos académicos suelen mezclar sintaxis formal con instrucciones descriptivas en lenguaje natural (ej. "combinar las dos mitades ordenadas"). El sistema debe ser capaz de *parsear* una sintaxis estricta pero flexible, y manejar estos casos mediante la integración con LLMs que interpreten el contexto semántico.
- **Análisis Multidimensional de Casos:** Un mismo algoritmo puede comportarse de manera radicalmente diferente según la distribución de la entrada:
 - **Peor caso (Big O):** Cota superior asintótica, garantiza el límite máximo de recursos.
 - **Mejor caso (Big Ω):** Cota inferior, representa el escenario más favorable.
 - **Caso promedio (Big θ):** Comportamiento esperado bajo distribuciones probabilísticas, requiriendo análisis con variables indicadoras según el Capítulo 5 de Cormen.
- El sistema debe identificar estructuras condicionales (`if/else`) que bifurcan el flujo y generan distintos escenarios de costo. Esto se evidencia en algoritmos de ordenamiento donde Quicksort presenta $O(n^2)$ en el peor caso, pero $\Theta(n \lg n)$ en promedio.

- **Resolución Matemática Rigurosa:** No basta con identificar los bucles o llamadas recursivas; el sistema debe ser capaz de:
 - Simplificar expresiones algebraicas complejas.
 - Resolver recurrencias que no se ajustan al Teorema Maestro básico (casos con $f(n)$ no polinómica).
 - Aplicar métodos alternativos como Akra-Bazzi, cambio de variables o funciones generatrices.
 - Manejar recurrencias no estándar como Fibonacci ($T(n) = T(n - 1) + T(n - 2)$) o Torres de Hanoi ($T(n) = 2T(n - 1) + 1$).
- **Patrones Especiales:** Algoritmos como la exponenciación rápida (*repeated squaring*) presentan múltiples ramas recursivas mutuamente excluyentes que, aunque superficialmente parecen generar complejidad exponencial, en realidad resultan en $\Theta(\lg n)$. La detección de estos patrones requiere análisis semántico del flujo de control.

3.3 Alcance del Sistema

El alcance de **EffiCode Analyzer** incluye:

Aspecto	Soportado	Detalle
Estructuras secuenciales	✓	Asignaciones, operaciones aritméticas
Estructuras iterativas	✓	for, while, repeat...until, anidados
Estructuras recursivas	✓	Lineal, binaria, múltiple, exponencial
Operadores especiales	✓	$\leftarrow$, { (asignación)}, $\leq$, $\geq$, $\neq$, $[x]$, $\lfloor x \rfloor$
Análisis de casos	✓	Peor, mejor y promedio

Visualización	✓	AST, árbol de recursión, costos por línea
---------------	---	---

3.4 Limitaciones

El sistema se limita a algoritmos que puedan expresarse mediante la gramática EBNF definida en el módulo de *parsing*, excluyendo:

- Algoritmos con estructuras de datos complejas no primitivas.
- Código con efectos secundarios o entrada/salida.
- Pseudocódigo con sintaxis no estándar o excesivamente informal.
- Análisis de complejidad espacial (memoria): Es más complejo detectar uso de memoria dinámicamente, el libro de Cormen enfatiza principalmente el análisis temporal, aunque representa una extensión natural para trabajos futuros.

4. Entrada de Datos al Sistema

El sistema está diseñado para procesar algoritmos escritos en pseudocódigo. La entrada de datos se gestiona a través de una interfaz web interactiva.

4.1. Formato y Sintaxis

El **EffiCode Analyzer** implementa un *Parser* personalizado (basado en la librería [Lark](#) y expresiones regulares) que acepta una sintaxis inspirada en el libro de Cormen (CLRS).

Según los archivos [Grammar.py](#) y [Parser.py](#), las características sintácticas soportadas incluyen:

- **Asignación:** Uso del símbolo $\leftarrow$ o `<-` (ej. $x \leftarrow 10$).
- **Bucles Iterativos:**
 - `for i ← start to end do ...` (traducido internamente a `range(start, end + 1)` de Python).
 - `for i ← start downto end do ...` (traducido a `range` con paso negativo).
 - `while condition do ...`
 - `repeat ... until condition`
- **Condicionales:** Estructuras `if ... then ... else` y `elseif`.
- **Operadores Matemáticos y Lógicos:** Soporte para $\leq$, $\geq$, $\neq$, `and`, `or`, `not`, así como funciones de piso y techo ($\lfloor x \rfloor$, $\lceil x \rceil$).
- **Arreglos:** Acceso mediante corchetes `A[i]`.

4.2. Mecanismo de Ingreso

La interacción con el usuario se realiza mediante el componente `CodeEditor.tsx` en el Frontend. Este editor ofrece:

- **Entrada Directa:** Un área de texto con resaltado de sintaxis (*syntax highlighting*) específico para las palabras clave del pseudocódigo (`keyword`, `builtin`, `operator`).
- **Asistencia por IA (Modalidad Lenguaje Natural):** Existe una funcionalidad integrada (`NaturalLanguageModal`) que permite al usuario describir un algoritmo en español (ej. "Genera un algoritmo de búsqueda binaria"), la cual invoca al servicio de LLM para traducir esa descripción a la sintaxis estricta de pseudocódigo requerida por el *parser*.
- **Formateo Automático:** Herramientas para indentar correctamente el código, crucial para el análisis de bloques basado en indentación (similar a Python).

4.3. Consideraciones sobre Sentencias en Lenguaje Natural

El sistema aborda la integración de lenguaje natural en el pseudocódigo de dos formas:

- **Comentarios formales:** Las líneas que inician con `//` son ignoradas por el *parser*, permitiendo anotaciones descriptivas sin afectar el análisis.
- **Traducción asistida por IA:** Mediante el botón "Lenguaje Natural", el usuario puede ingresar descripciones informales que son convertidas automáticamente a pseudocódigo estructurado por el LLM. Esta funcionalidad incluye una advertencia indicando que el resultado es generado por IA y puede requerir verificación.
- **Funciones abstractas:** Llamadas como `MERGE(A, p, q, r)` representan operaciones cuyo costo se asume constante o se especifica según el contexto del algoritmo.

5. Estrategia Algorítmica y Técnica

Para determinar la complejidad, el sistema no "ejecuta" el código con un cronómetro (*profiling*), sino que realiza un **Análisis Estático y Simbólico**. Esta estrategia se fundamenta en el paradigma de análisis de código fuente sin ejecución, analizando la estructura sintáctica y semántica del algoritmo.

La estrategia se divide en dos ramas principales gestionadas por el Backend, dependiendo de si el algoritmo es iterativo o recursivo.

5.1. Traducción a AST (Abstract Syntax Tree)

El primer paso fundamental es convertir el pseudocódigo en una estructura de árbol manipulable

programáticamente.

Proceso de Transformación:

1. **Validación Gramatical:** La clase `Parser (Parser.py)` recibe el texto crudo del pseudocódigo y realiza una validación sintáctica utilizando el motor de parsing `Lark` a través del servicio `Grammar (Grammar.py)`. La gramática EBNF define las construcciones válidas del pseudocódigo estilo Corman.
2. **Transpilación a Python:** El `Parser` implementa un traductor local robusto que convierte el pseudocódigo a código Python semánticamente equivalente. Esta traducción preserva la estructura de control (bucles, condicionales, llamadas recursivas) sin depender de servicios externos.
3. **Generación del AST:** Utilizando el módulo nativo `ast` de Python, se genera el Árbol de Sintaxis Abstracta. El servicio `AST (Ast.py)` encapsula este árbol, permitiendo recorrerlo y analizarlo programáticamente.

Ejemplo de Transformación:

Pseudocódigo	Python equivalente
FOR i = 1 TO n	for i in range(1, n+1):
x = x + 1	x = x + 1

5.2. Análisis Iterativo (Patrón Visitor)

Para algoritmos iterativos, se utiliza la clase `EfficiencyVisitor (EfficiencyVisitor.py)`, que implementa el Patrón de Diseño *Visitor* heredando de `ast.NodeVisitor`.

Componentes del Análisis:

5.2.1. Recorrido del Árbol

El visitante recorre sistemáticamente cada nodo del AST, identificando:

- **Operaciones elementales:** Asignaciones, operaciones aritméticas, comparaciones.
- **Estructuras de control:** Bucles (`FOR`, `WHILE`), condicionales (`IF/ELSE`).
- **Llamadas a funciones:** Para detectar posible recursión.

5.2.2. Cálculo Simbólico con SymPy

Se utiliza la librería [SymPy](#) para manejar expresiones matemáticas simbólicas. Esto permite:

- Representar variables como símbolos matemáticos (n, i, j) .
- Realizar simplificaciones algebraicas automáticas.
- Evaluar expresiones de forma exacta sin aproximaciones numéricas.

5.2.3. Manejo de Sumatorias (Σ)

Cuando el visitor encuentra un bucle **FOR**, envuelve el costo de su cuerpo en una sumatoria:

$$T_{bucle} = \sum_{variable=inicio}^{fin} T_{cuerpo}$$

Ejemplo de bucles anidados dependientes:

FOR $i = 1$ **TO** n

FOR $j = i$ **TO** n

operación con costo c

Se genera la sumatoria:

$$T(n) = \sum_{i=1}^n \sum_{j=i}^n c = \sum_{i=1}^n c(n-i+1) = c \frac{n(n+1)}{2} \in \Theta(n^2)$$

5.2.4. Ramificación (If/Else)

Para los condicionales, el sistema calcula el costo de ambas ramas y utiliza:

- **Peor caso:** $T_{worst}(n) = \max(T_{if}, T_{else})$
- **Mejor caso:** $T_{best}(n) = \min(T_{if}, T_{else})$

Esto permite construir las cotas superiores (O) e inferior (Ω) de manera independiente.

5.3. Análisis Recursivo y Resolución de Recurrencias

Si el sistema detecta llamadas recursivas, delega el problema al [RecurrenceSolver](#) ([RecurrenceSolver.py](#)), que implementa los 7 métodos del libro "Introduction to Algorithms" (Cormen et al., 4th Ed).

5.3.1. Detección de Parámetros

El solver identifica automáticamente los parámetros de la recurrencia:

- a : Número de subproblemas (llamadas recursivas).
- b : Factor de división del problema.
- $f(n)$: Costo de combinación/trabajo fuera de las llamadas recursivas.

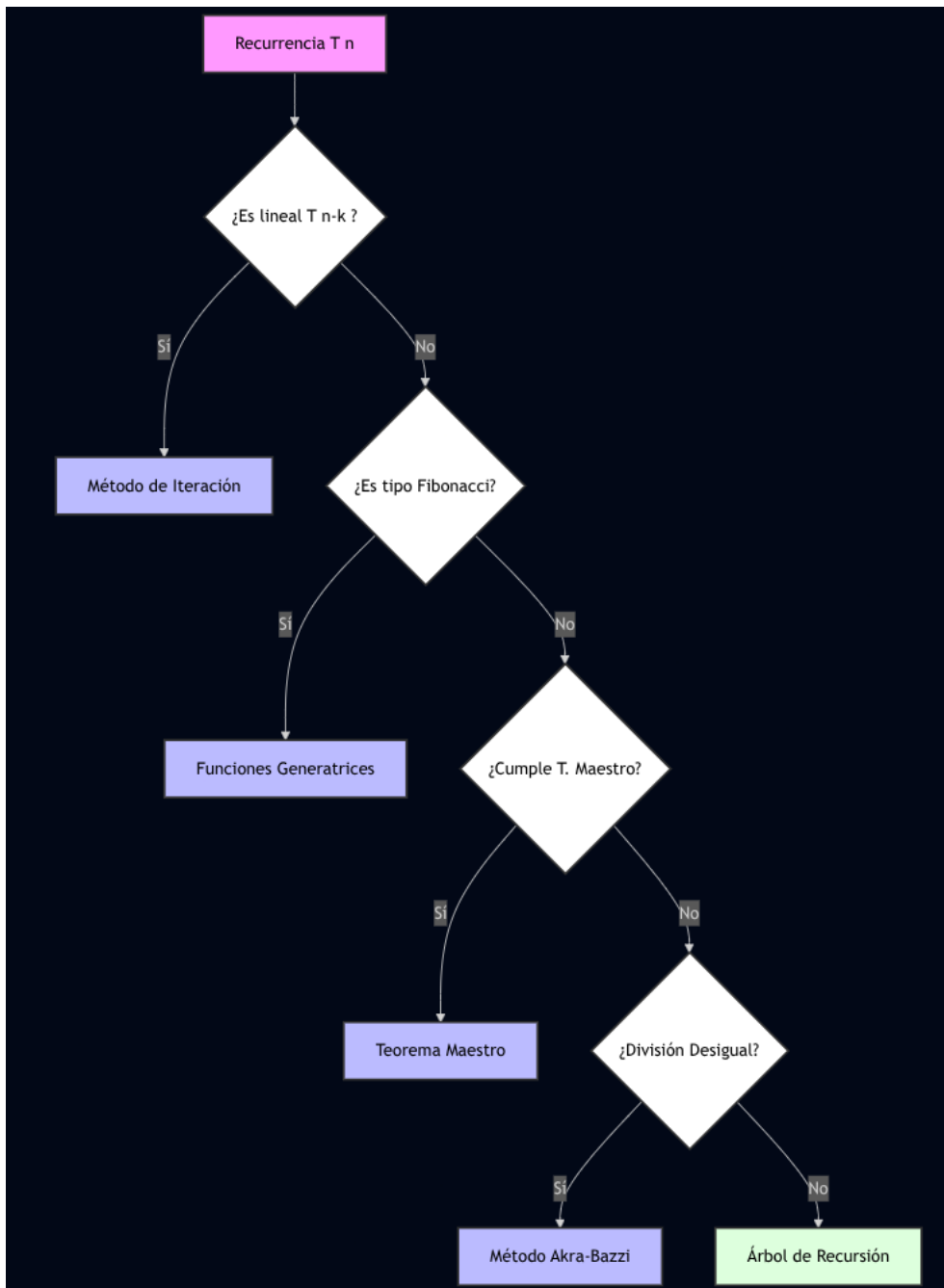
Forma estándar:

$$T(n) = a T(n/b) + f(n)$$

5.3.2. Métodos de Resolución Implementados

Método	Aplicabilidad	Ejemplo
Teorema Maestro	$T(n) = aT(n/b) + f(n)$ con condiciones específicas	Merge Sort, Binary Search
Árbol de Recursión	Cuando el T. Maestro no aplica directamente	Visualización de costos por nivel
Método de Iteración	Recurrencias lineales $T(n) = T(n - k) + f(n)$	Factorial, búsqueda lineal recursiva
Akra-Bazzi	División desigual (subproblemas de distintos tamaños)	$T(n) = T(n/3) + T(2n/3) + n$
Funciones Generatrices	Recurrencias tipo Fibonacci	$T(n) = T(n - 1) + T(n - 2)$
Cambio de Variables	Transformación a forma estándar	$T(n) = 2 T(\sqrt{n}) + \log n$
Sustitución	Verificación de hipótesis inductiva	Demostración formal de cotas

5.3.3. Flujo de Selección de Algoritmo (Algorithm Selection)



5.4. Justificación Matemática Paso a Paso

Una característica distintiva del sistema es que no solo devuelve el resultado final (O, Ω, Θ), sino que genera una justificación matemática completa con:

1. **Identificación de parámetros:** "Se detectaron $a = 2$ subproblemas, $b = 2$ factor de división, $f(n) = n$ ".
2. **Selección del método:** "Aplicando el Teorema Maestro...".
3. **Verificación de condiciones:** "Caso 2: $f(n) = \Theta(n^{\log_b a}) = \Theta(n^1)$ ".
4. **Resultado final:** " $T(n) = \Theta(n \lg n)$ ".

Esta estrategia híbrida garantiza que el sistema pueda manejar desde un simple bucle **FOR** hasta algoritmos complejos como *Merge Sort*, *QuickSort* o *Fibonacci*, proporcionando siempre una justificación teórica rigurosa basada en los fundamentos del curso de Análisis y Diseño de Algoritmos.

6. Arquitectura e Implementación del Sistema

6.1. Patrón Arquitectónico Adoptado

El sistema sigue una arquitectura **Cliente-Servidor (Client-Server)** fuertemente desacoplada, utilizando el patrón **REST API** para la comunicación entre capas.

Capa	Tecnología	Responsabilidad
Frontend (Cliente)	React 19 + Vite 7 + TypeScript	Presentación, captura de datos, visualización interactiva.
Backend (Servidor)	Python 3.11 + FastAPI	Lógica de negocio, <i>parsing</i> , análisis matemático, Inteligencia Artificial (ML/LLM) .

Características de la comunicación:

- **Protocolo:** HTTP/HTTPS
- **Formato de datos:** JSON

- **Estilo:** RESTful con endpoints semánticos
- **Puerto por defecto:** Backend en :8000, Frontend en :5173

6.2. Justificación del Diseño

Esta separación se eligió por razones técnicas fundamentales:

6.2.1. Especialización del Lenguaje

Aspecto	Python (Backend)	TypeScript (Frontend)
Fortaleza	Computación simbólica, científica y ML	UI interactiva y responsiva
Librerías clave	sympy, ast, numpy, lark-parser	React, Monaco Editor, MathJax
Justificación	Estándar en academia, análisis matemático y ML	Mejor experiencia de usuario en navegadores

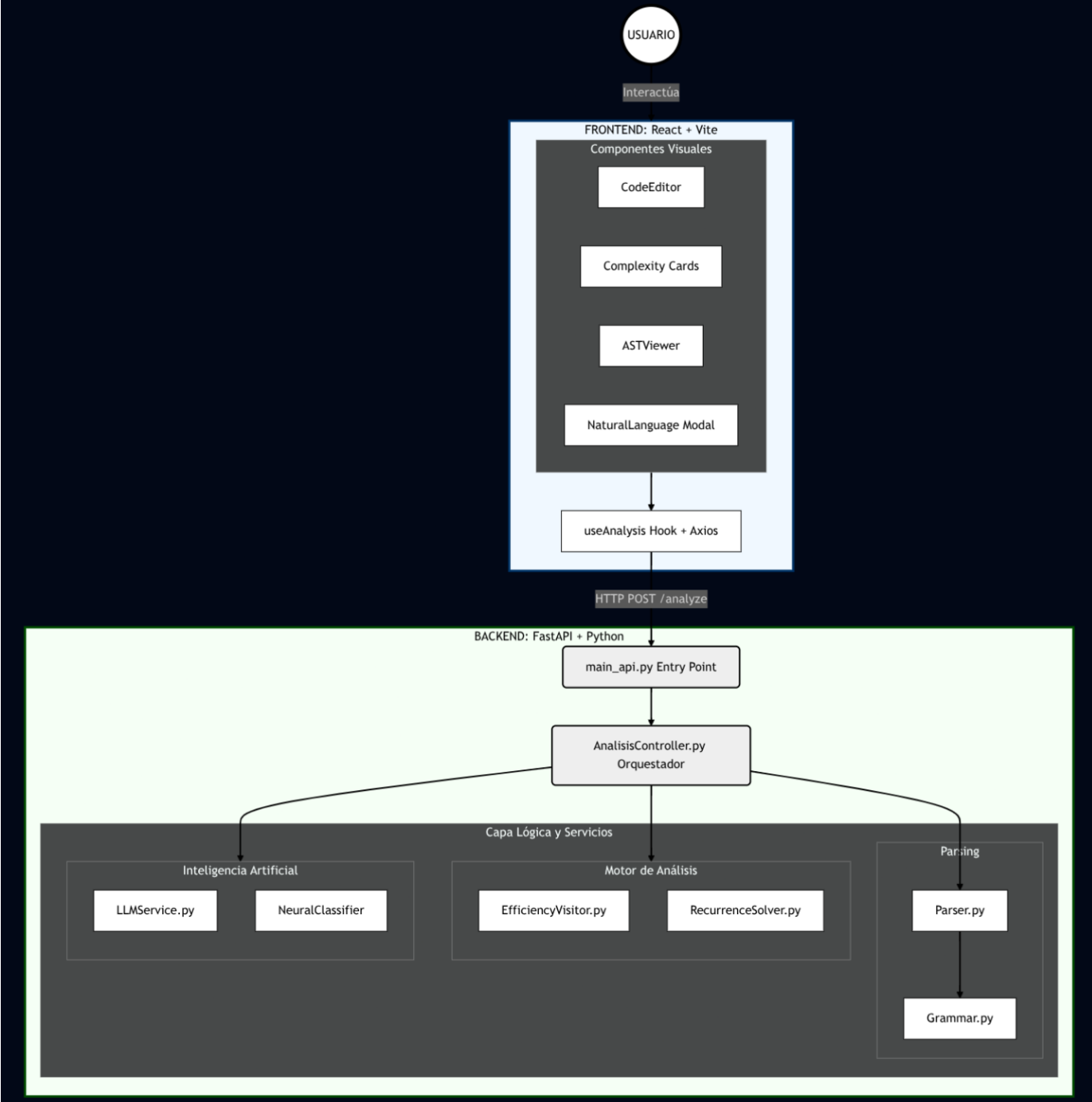
6.2.2. Escalabilidad y Mantenibilidad

- **Escalabilidad horizontal:** El Backend puede desplegarse en múltiples instancias o contenedores.
- **Independencia de despliegue:** Frontend puede ser hospedado como sitio estático (CDN), Backend en servidor dedicado.
- **Facilidad de testing:** Cada capa puede probarse de forma independiente.

6.3. Diagrama de Arquitectura

Nota: Incluir aquí la imagen del archivo [PROYECTO/DIAGRAMAS/Diagrama de Arquitectura.pdf](#).

A continuación, se presenta la representación lógica actualizada incluyendo el módulo de IA:



6.4. Componentes del Sistema

6.4.1. Capa de Presentación (Frontend)

Componente	Ubicación	Responsabilidad
CodeEditor	src/components/editor /	Editor de pseudocódigo con resaltado sintáctico.
NaturalLanguageModal	src/components/editor /	Modal para traducción de lenguaje natural.
ComplexityCards	src/components/analysis/	Visualización de O , Ω , Θ .
ResolutionSteps	src/components/analysis/	Pasos matemáticos de resolución.
AIValidation	src/components/analysis/	Resultado de validación por IA.
useAnalysis	src/hooks/	Hook personalizado para gestión de estado.

6.4.2. Capa de API (Backend)

Componente	Ubicación	Responsabilidad

main_api.py	/Backend/	Entry point, configuración CORS, lifespan.
analysis.py (router)	/api/routers/	Endpoints REST y validación de esquemas.
deps.py	/api/	Contenedor de dependencias (Dependency Injection).

6.4.3. Capa de Servicios (Lógica de Negocio)

Componente	Ubicación	Responsabilidad
Parser.py	/Modelos/	Transpilación pseudocódigo (→) Python (→) AST.
Grammar.py	/Servicios/	Gramática EBNF con motor Lark.
EfficiencyVisitor.py	/Servicios/	Análisis iterativo con SymPy (Patrón Visitor).
RecurrenceSolver.py	/Servicios/	Resolución de recurrencias (7 métodos Cormen).
LLMService.py	/Servicios/	Integración con Google Gemini API.

6.4.4. Módulo de Redes Neuronales (NeuralClassifier)

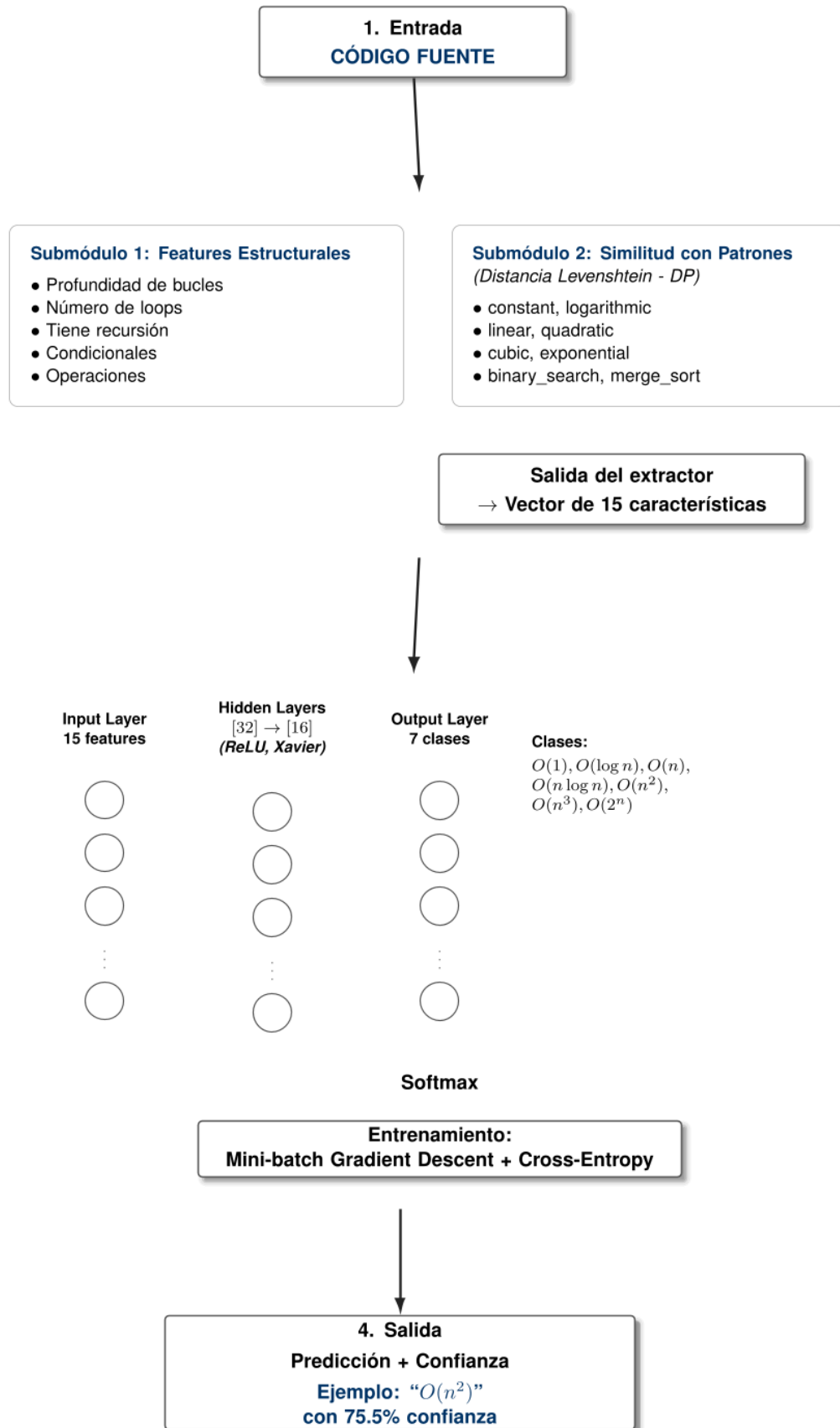
Este módulo implementa un **Perceptrón Multicapa (MLP)** desde cero utilizando únicamente **NumPy**, sin dependencia de frameworks de alto nivel como TensorFlow o PyTorch. Es uno de los componentes más avanzados del sistema, diseñado para realizar una clasificación probabilística rápida de la complejidad.

Componente	Archivo	Responsabilidad
NeuralComplexityClassifier	classifier.py	Patrón Facade: Interfaz unificada del clasificador.
NeuralNetwork	model.py	Red neuronal MLP con <i>backpropagation</i> .
FeatureExtractor	features.py	Extracción de características del código AST.
HyperTuner	tuner.py	Optimización de arquitectura con Backtracking .
consts.py	consts.py	Constantes y patrones de referencia.

Arquitectura de la Red Neuronal:

- **Entrada:** Vector de 15 características (profundidad, bucles, recursión, métricas de Levenshtein).
- **Capas Ocultas:** Configuración dinámica optimizada (ej. [32] -> [16]) con activación **ReLU**.
- **Salida:** Capa *Softmax* con 7 neuronas (una por clase de complejidad).
- **Entrenamiento:** *Mini-batch Gradient Descent* + *Cross-Entropy Loss*.

CLASIFICADOR NEURAL DE COMPLEJIDAD



Clases de Complejidad Soportadas:

Índice	Complejidad	Descripción
0	$O(1)$	Constante
1	$O(\log n)$	Logarítmica
2	$O(n)$	Lineal
3	$O(n \log n)$	Linealítmica
4	$O(n^2)$	Cuadrática
5	$O(n^3)$	Cúbica
6	$O(2^n)$	Exponencial

Algoritmos y Técnicas Implementados en el Módulo:

- **Programación Dinámica:** Implementada en [features.py](#) para el cálculo de la Distancia de Levenshtein (similitud de patrones).
- **Backtracking con Poda:** Implementado en [tuner.py](#) para la búsqueda de hiperparámetros óptimos.
- **Algoritmo Greedy:** Descenso de gradiente en [model.py](#).
- **Inicialización Xavier/Glorot:** Para convergencia estable de pesos.

6.5. Flujo de Datos y Lógica Interna

El sistema implementa un **Análisis Dual**, combinando precisión matemática con inferencia estadística.

1. **INGRESO:** El usuario envía el pseudocódigo (**POST /analyze**).
2. **VALIDACIÓN & PARSING:** Se valida la gramática y se genera el AST.
3. **CLASIFICACIÓN:** Se determina si es Iterativo o Recursivo.
4. **ANÁLISIS DUAL (Paralelo):**
 - **Rama A (Simbólica):** **EfficiencyVisitor** o **RecurrenceSolver** realizan la deducción matemática rigurosa.
 - **Rama B (Neural):** **NeuralComplexityClassifier** extrae *features*, ejecuta el *forward pass* en el MLP y predice la clase de complejidad con un porcentaje de confianza (ej. "94.5% $O(n^2)$ ").
5. **VALIDACIÓN IA (Opcional):** **LLMService** consulta a Gemini para explicaciones semánticas.
6. **RESPUESTA:** Se combinan los resultados en un objeto JSON.

1. INGRESO

Usuario escribe pseudocódigo → Click 'Analizar'

Envía: `POST /analyze { "code": "INSERTION-SORT(A,n)..." }`

2. VALIDACIÓN

Pydantic valida schema → `Grammar.py` valida sintaxis (Lark)

Si error → HTTP 422 con mensaje detallado

3. PARSING

`Parser.py` transpila a Python → módulo `ast` genera el AST

4. CLASIFICACIÓN

`Ast.extraer_funciones()` → Detectar llamadas recursivas

¿Recursivo? → `TipoAlgoritmo.RECURSIVO` : `TipoAlgoritmo.ITERATIVO`

5. ANÁLISIS (DUAL)

ANÁLISIS SIMBÓLICO

- `EfficiencyVisitor` (iter)
 - `RecurrenceSolver` (recur)
- Análisis matemático riguroso

CLASIFICACIÓN NEURAL

- `NeuralComplexityClassifier`:
 1. Extraer features del código
 2. Forward pass por MLP
 3. Softmax → probabilidades
- Clasificación rápida

6. VALIDACIÓN IA (Opcional)

`LLMService` consulta Google Gemini para validar el resultado

7. RESPUESTA

Se construye `ResultadoAnálisis` (DTO) → JSON Response

6.6. Endpoints de la API REST

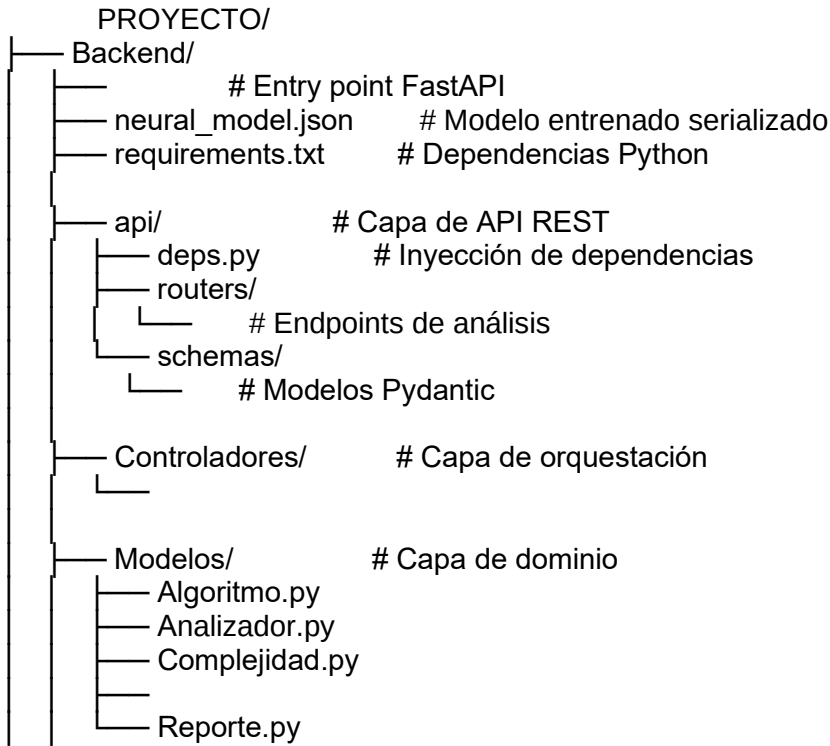
Método	Endpoint	Descripción	Request Body	Response
POST	/analyze	Análisis de complejidad	{ code: string }	AnalysisResponse
POST	/validate	Validación con IA	{ code, complexity }	ValidationResponse
POST	/translate/...	Traducción lenguaje natural	{ text: string }	{ pseudocode: string }
POST	/report/pdf	Generación de reporte	ReportRequest	application/pdf

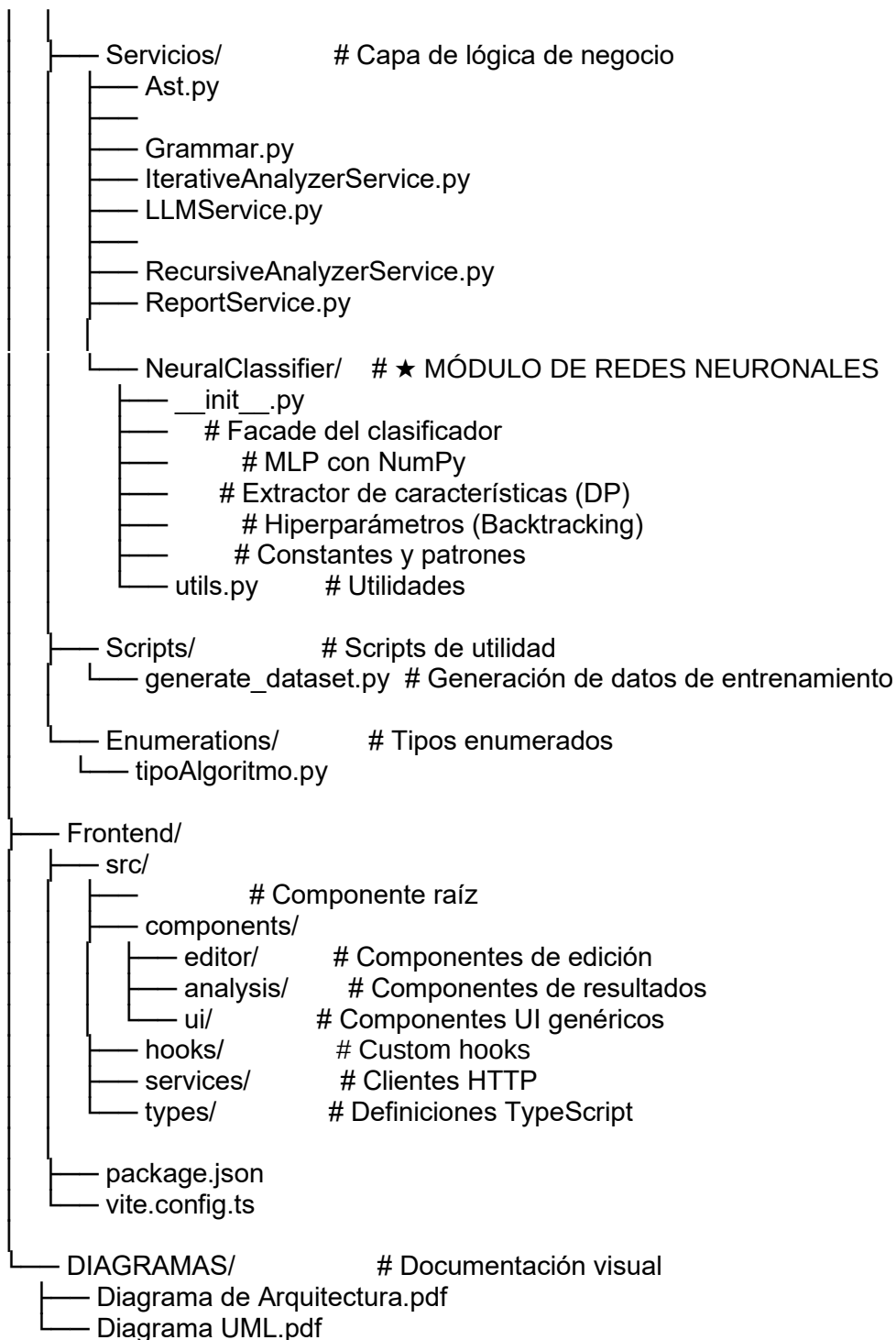
6.7. Manejo de Errores

Nivel	Excepción	Causa	HTTP	Mensaje al Usuario
Sintaxis	SyntaxError	Pseudocódigo inválido	422	Línea y descripción del error
Parsing	ValueError	Fallo al generar	400	"Error al procesar"

	r	AST		el código"
Análisis	AnalysisError	Recurrencia no estándar	200	"Complejidad indeterminada" + fallback IA
LLM	HTTPException	API Gemini caída	503	"Servicio IA no disponible"
Interno	Exception	Error no controlado	500	"Error interno del servidor"

6.8. Estructura del Código Fuente





7. Integración de Modelos de Lenguaje (LLMs) e Inteligencia Artificial

El sistema **EffiCode Analyzer** incorpora dos capas de inteligencia artificial complementarias: un modelo

de lenguaje externo (LLM) para procesamiento de lenguaje natural y un clasificador neuronal local para reconocimiento de patrones.

7.1. Modelo y Tecnología

7.1.1. LLM Externo: Google Gemini

El proyecto integra la API de Google Gemini a través de la clase `LLMService` (`LLMService.py`).

Aspecto	Detalle
Proveedor	Google AI
Modelo	gemini-2.5-pro (configurable)
SDK	google-genai (versión > 1.0.0)
Autenticación	Variable de entorno GOOGLE_API_KEY

Características del servicio:

```
class LLMService:
    """
    Servicio principal de IA con el nuevo SDK google-genai.
    Opera con el contexto permanente del libro "Introduction to Algorithms".
    """

    def __init__(self, modelo: str = "gemini-2.5-pro"):
        self._api_key = os.getenv("GOOGLE_API_KEY")
```

```
self.client = genai.Client(api_key=self._api_key)

self._modelo = modelo

self._libro_contexto_file = self._gestionar_cache_libro()
```

7.1.2. Contexto Académico Permanente

Una característica distintiva del sistema es que todas las consultas al LLM incluyen el libro *"Introduction to Algorithms"* de Cormen como contexto. Esto garantiza que las respuestas sean académicamente rigurosas y consistentes con la terminología del curso.

Sistema de Caché Inteligente:

El sistema implementa una lógica de caché para optimizar el uso de la API y reducir la latencia:

1. **Verificación Local:** Se consulta el archivo `cache_file.json` que contiene el `file_id`, nombre y fecha de subida.
2. **Validación de Expiración:** Se verifica si el caché tiene menos de 48 horas de antigüedad.
3. **Consulta a Google:**
 - **Si existe y es válido:** Se reutiliza el archivo ya subido a Google (Estado: `ACTIVE`). Esto ahorra tiempo y costos de procesamiento.
 - **Si no existe o expiró:** Se sube nuevamente el PDF del libro (~80MB) a la API de Google Files y se actualiza la referencia en el caché local.

Beneficio: Esta estrategia evita subir un archivo pesado en cada reinicio del servidor o con cada consulta, mejorando drásticamente el rendimiento.

7.1.3. Clasificador Neural Local

Además del LLM externo, el sistema cuenta con un **Perceptrón Multicapa (MLP)** implementado desde cero para clasificación rápida de patrones algorítmicos (ver sección 6.4.4).

Aspecto	LLM (Gemini)	Neural Local (MLP)
Ubicación	API externa (nube)	Local (<code>NumPy</code>)

Latencia	~1-3 segundos	~10-50 ms
Costo	Por tokens/request	Gratuito
Uso principal	Validación, traducción NL	Clasificación rápida
Dependencia de red	Requiere internet	Sin conexión

7.2. Funciones Asistidas por IA

Los LLMs no reemplazan el análisis matemático, sino que lo complementan en momentos específicos del flujo.

7.2.1. Traducción Lenguaje Natural → Pseudocódigo

- **Método:** `traducir_natural_a_pseudocodigo(texto: str)`

Permite al usuario describir un algoritmo en español y obtener pseudocódigo estilo Cormen listo para analizar.

Entrada (Lenguaje Natural)	Salida (Pseudocódigo Cormen)
"Ordena una lista comparando elementos adyacentes"	<code>BUBBLE-SORT(A, n)</code> con bucles y <code>exchange</code>
"Busca un elemento dividiendo el arreglo a la mitad"	<code>BINARY-SEARCH(A, v, low, high)</code> con comparaciones
"Encuentra el camino más corto en un grafo"	<code>DIJKSTRA(G, w, s)</code> con cola de prioridad

Prompt Engineering aplicado:

```
prompt = f"""
```

Actúa como un generador de pseudocódigo del libro 'Introduction to Algorithms'.

****REGLAS Estrictas:****

1. ****SOLO CÓDIGO:**** SIN explicaciones, SIN títulos, SIN markdown

2. ****Formato exacto del libro:****

- Usa ``←`` para asignaciones (NO ``=``)

- Usa ``for ... to ... do``, ``while ... do``, ``if ... then``

- Usa ``exchange A[i] with A[j]`` para intercambios

3. ****Indentación:**** 4 espacios por nivel

```
... """
```

7.2.2. Traducción Pseudocódigo → Python

- **Método:** `traducir_pseudocodigo_a_python(pseudocodigo: str)`

Convierte pseudocódigo estilo Cormen a código Python ejecutable, adaptando:

- Índices de 1-indexado a 0-indexado.
- Símbolos especiales ($\leftrightarrow$, $\leq$)
- Estructuras de control a sintaxis Python.

7.2.3. Validación y Explicación de Complejidad

- **Método:** `validar_analisis(complejidad: Complejidad, pseudocodigo: str)`

Cuando el motor matemático deduce una complejidad, se envía al LLM para obtener una "segunda opinión experta".

Flujo de validación:

1. **Motor Matemático (RecurrenceSolver):** Calcula el resultado (ej. $O(n^2)$) y genera la justificación matemática.

2. **Envío al LLM:** Se envía el código y el resultado calculado junto con un prompt de rol ("Como profesor de algoritmos...").
3. **Respuesta del LLM:** Confirma si el análisis es correcto o sugiere correcciones, proporcionando una explicación semántica ("El algoritmo tiene dos bucles anidados...").
4. **Comparación:** El sistema compara ambas salidas para determinar el nivel de confianza.

7.2.4. Clasificación de Paradigmas de Diseño

- **Método:** `clasificar_patron(algoritmo: Algoritmo)`

Identifica el paradigma algorítmico principal:

- Divide y Vencerás (*Divide and Conquer*)
- Programación Dinámica (*Dynamic Programming*)
- Algoritmos Voraces (*Greedy Algorithms*)
- *Backtracking*
- *Branch and Bound*

7.3. Validación de Confiabilidad y Mitigación de Alucinaciones

El sistema implementa una estrategia de prioridad del análisis matemático para mitigar las conocidas "alucinaciones" de los LLMs.

7.3.1. Jerarquía de Confianza

1. **ANÁLISIS SIMBÓLICO (Máxima prioridad):**
 - `RecurrenceSolver` + `EfficiencyVisitor`
 - Cálculo matemático riguroso basado en Cormen.
 - Su resultado siempre se muestra como el principal.
2. **CLASIFICADOR NEURAL (Apoyo local):**
 - `NeuralComplexityClassifier`
 - Clasificación rápida por patrones estructurales.
 - Útil para validación cruzada.
3. **LLM EXTERNO (Enriquecimiento):**
 - Google Gemini
 - Validación semántica y explicaciones textuales.
 - Nunca sobrescribe el resultado matemático sin advertencia.

7.3.2. Indicadores de Confianza en la Interfaz

El componente `ValidationCard.tsx` del Frontend muestra el nivel de confianza basado en la concordancia

entre las fuentes:

Escenario	Indicador	Mensaje
Motor = LLM	Alta Confianza	"Análisis verificado por IA"
Motor $\neq$ LLM	Revisar Manualmente	"Discrepancia detectada - Se muestran ambos análisis"
LLM no disponible	Sin Validación IA	"Resultado basado únicamente en análisis matemático"
Error de API	Error de Conexión	"No se pudo contactar el servicio de IA"

7.3.3. Estrategias de Mitigación

Estrategia	Implementación
Contexto Autoritativo	Todas las consultas incluyen el libro de Cormen como fuente de verdad.
Prompts Restrictivos	Instrucciones explícitas de formato y contenido negativo.
Post-procesamiento	Limpieza de markdown, títulos y explicaciones no solicitadas.
Prioridad Matemática	El resultado del RecurrenceSolver nunca es sobrescrito automáticamente.

Transparencia	Siempre se muestra al usuario la fuente de cada resultado.
---------------	--

7.4. Arquitectura de Integración

La arquitectura integra los servicios de IA como módulos independientes dentro del Backend:

- **LLMService.py**: Fachada para la API de Google Gemini. Maneja la autenticación, el caché de archivos y la construcción de prompts.
- **NeuralClassifier/**: Módulo local de redes neuronales que opera sin conexión a internet.

7.5. Consideraciones de Seguridad y Costos

Aspecto	Medida Implementada
API Key	Almacenada en variable de entorno (.env), nunca hardcodeada en código.
Rate Limiting	El sistema de caché de archivos reduce drásticamente las llamadas pesadas de subida a la API.
Fallback	Si la API falla o excede la cuota, el análisis matemático continúa funcionando sin interrupción.
Costos	La caché de 48h evita re-subir el libro (80+ MB) repetidamente, optimizando el uso de ancho de banda y almacenamiento.
Privacidad	El código del usuario se envía a Google únicamente para el propósito del análisis y bajo los términos de uso de la API Enterprise (si aplica).

8. Análisis de Eficiencia del Sistema

Debido a la rigurosidad matemática y la extensión requerida para analizar exhaustivamente la complejidad algorítmica del propio sistema desarrollado (**EffiCode Analyzer**), este análisis se presenta como un documento técnico independiente.

El estudio detallado incluye la derivación formal de las complejidades para los módulos de *Parsing*, *Análisis Iterativo* y *Resolución de Recurrencias*, así como la evaluación empírica de tiempos de ejecución y comparativas de desempeño.

Referencia del documento:

El análisis completo se encuentra adjunto en los anexos del proyecto bajo la siguiente ruta:

Ubicación: Backend/Documentacion/Diagramas/

Nombre del archivo: 8.Análisis de Eficiencia del Sistema EffiCode Analyzer.pdf

Este documento aborda en profundidad:

1. **Complejidad Asintótica:** Derivación matemática (O , Ω , Θ) de los componentes internos.
2. **Evaluación Empírica:** Pruebas de estrés y mediciones de tiempo real.
3. **Comparativas:** Análisis de precisión frente a soluciones manuales y modelos generativos (LLMs)

9. Casos de Prueba

Este capítulo presenta los casos de prueba utilizados para validar el correcto funcionamiento del sistema **EffiCode Analyzer**. Los algoritmos fueron extraídos del archivo [Algoritmos de prueba.txt](#) y del conjunto de entrenamiento [training_algorithms.json](#), abarcando todas las clases de complejidad soportadas.

9.1. Metodología de Pruebas

Proceso de validación:

1. Se ingresa el pseudocódigo en el sistema.
2. Se ejecuta el análisis automático.
3. Se compara el resultado con la complejidad teórica conocida.
4. Se verifica que la justificación matemática sea correcta.

Criterios de éxito:

- Complejidad correcta en notación O , Ω y Θ .
- Método de resolución apropiado (Teorema Maestro, Sumatorias, etc.).

- Justificación paso a paso coherente.

9.2. Algoritmos de Complejidad Constante - $O(1)$

Caso 1: CONSTANT-ACCESS

Pseudocódigo:

```
CONSTANT-ACCESS(A, i)
```

```
    return A[i]
```

Aspecto	Valor
Tipo	Iterativo (operación simple)
Esperado	$\Theta(1)$
Resultado Sistema	$\Theta(1)$
Método usado	Análisis de operaciones elementales

Justificación del sistema:

"Acceso directo a memoria por índice. Tiempo constante independiente del tamaño de la entrada."

Caso 2: SWAP

Pseudocódigo:

```
SWAP(A, i, j)
```

```
    temp  $\leftarrow$  A[i]
```

```
    A[i]  $\leftarrow$  A[j]
```

$A[j] \leftarrow \text{temp}$

Aspecto	Valor
Tipo	Iterativo
Esperado	$\Theta(1)$
Resultado Sistema	$\Theta(1)$
Función de costo	$T(n) = c_1 + c_2 + c_3 = c$

9.3. Algoritmos de Complejidad Logarítmica - $O(\log n)$

Caso 3: BINARY-SEARCH (Recursivo)

Pseudocódigo:

BINARY-SEARCH(A, p, r, x)

if $p > r$ then

return -1

$q \leftarrow (p + r) \div 2$

if $A[q] = x$ then

return q

else if $A[q] > x$ then

return BINARY-SEARCH(A, p, q - 1, x)

else

return BINARY-SEARCH(A, q + 1, r, x)

Aspecto	Valor
Tipo	Recursivo
Esperado	$\Theta(\log n)$
Resultado Sistema	$\Theta(\log n)$
Ecuación detectada	$T(n) = T(n/2) + \Theta(1)$
Método usado	Teorema Maestro (Caso 2)

Justificación del sistema:

1. **Ecuación de Recurrencia:** $T(n) = 1 \cdot T(n/2) + \Theta(1)$.
2. **Paso 1:** Identificar parámetros: $a = 1$, $b = 2$, $f(n) = \Theta(1)$.
3. **Paso 2:** Calcular $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$.
4. **Paso 3:** Comparar $f(n) = \Theta(n^{\log_b a}) : f(n) = \Theta(1) = \Theta(n^0)$
5. **Conclusión:** Caso 2 del Teorema Maestro aplicable ($k = 0$). Resultado: $T(n) = \Theta(\log n)$.

Caso 4: ITERATIVE-BINARY-SEARCH

Pseudocódigo:

ITERATIVE-BINARY-SEARCH(A, n, x)

low $\leftarrow$ 1

high $\leftarrow$ n

while low $\leq$ high do

mid $\leftarrow$ (low + high) div 2

if A[mid] = x then

return mid

```

else if A[mid] < x then
    low ← mid + 1
else
    high ← mid - 1
return -1

```

Aspecto	Valor
Tipo	Iterativo (while)
Esperado	$\Theta(\log n)$
Resultado Sistema	$\Theta(\log n)$
Análisis	El rango se reduce a la mitad en cada iteración

Caso 5: POWER (Exponenciación Rápida)

Pseudocódigo:

```

POWER(x, n)
    if n = 0 then
        return 1
    if n mod 2 = 0 then
        half ← POWER(x, n div 2)
        return half * half
    else
        return x * POWER(x, n - 1)

```

Aspecto	Valor
Tipo	Recursivo
Esperado	$\Theta(\log n)$
Resultado Sistema	$\Theta(\log n)$
Patrón detectado	Exponenciación por cuadrados repetidos

9.4. Algoritmos de Complejidad Lineal - $O(n)$

Caso 6: LINEAR-SEARCH

Pseudocódigo:

LINEAR-SEARCH(A, n, x)

 for $i \leftarrow 1$ to n do

 if $A[i] = x$ then

 return i

 return -1

Aspecto	Valor
Tipo	Iterativo
Esperado	Peor: $O(n)$, Mejor: $\Omega(1)$

Resultado Sistema	Peor: $O(n)$, Mejor: $\Omega(1)$
Función de costo	$T(n) = \sum_{i=1}^n c = c \cdot n$

Caso 7: FACTORIAL-ITERATIVE

Pseudocódigo:

FACTORIAL-ITERATIVE(n)

 result $\leftarrow$ 1

 for i $\leftarrow$ 2 to n do

 result $\leftarrow$ result * i

 return result

Aspecto	Valor
Tipo	Iterativo
Esperado	$\Theta(n)$
Resultado Sistema	$\Theta(n)$
Sumatoria	$T(n) = c_1 + \sum_{i=2}^n c_2 = c_1 + c_2(n - 1) = \Theta(n)$

Caso 8: FIND-MIN-MAX

Pseudocódigo:

FIND-MIN-MAX(A, n)

min $\leftarrow$ A[1]

max $\leftarrow$ A[1]

for i $\leftarrow$ 2 to n do

 if A[i] < min then

 min $\leftarrow$ A[i]

 if A[i] > max then

 max $\leftarrow$ A[i]

return min, max

Aspecto	Valor
Tipo	Iterativo con condicionales
Esperado	$\Theta(n)$
Resultado Sistema	$\Theta(n)$
Análisis	Bucle simple, condicionales no afectan la clase

9.5. Algoritmos de Complejidad Linealítmica - $O(n \log n)$

Caso 9: MERGE-SORT

Pseudocódigo:

MERGE-SORT(A, p, r)

```

if p < r then
    q ← (p + r) div 2
    MERGE-SORT(A, p, q)
    MERGE-SORT(A, q + 1, r)
    MERGE(A, p, q, r)

```

Aspecto	Valor
Tipo	Recursivo (Divide y Vencerás)
Esperado	$\Theta(n \log n)$
Resultado Sistema	$\Theta(n \log n)$
Ecuación detectada	$T(n) = 2 T(n/2) + \Theta(n)$
Método usado	Teorema Maestro (Caso 2)

Justificación del sistema:

1. **Ecuación de Recurrencia:** $T(n) = 2T(n/2) + \Theta(n)$
2. **Paso 1:** Identificar parámetros: $a = 2$, $b = 2$, $f(n) = \Theta(n)$ (trabajo de MERGE).
3. **Paso 2:** Calcular $n^{\log_b a} = n^{\log_2 2} = n^1 = n$
4. **Paso 3:** Comparar $f(n)$ con $n^{\log_b a}$: $f(n) = \Theta(n) = \Theta(n^1)$ Crecen al mismo ritmo polinómico.
5. **Conclusión:** Caso 2 del Teorema Maestro aplicable ($k = 0$). Resultado: $T(n) = \Theta(n \lg n)$

9.6. Algoritmos de Complejidad Cuadrática - $\Theta(n^2)$

Caso 10: BUBBLE-SORT

Pseudocódigo:

BURBUJA-SORT(A, n)

for i $\leftarrow$ 1 to n - 1 do

for j $\leftarrow$ n downto i + 1 do

if A[j] < A[j-1] then

temp $\leftarrow$ A[j]

A[j] $\leftarrow$ A[j-1]

A[j-1] $\leftarrow$ temp

Aspecto	Valor
Tipo	Iterativo (bucles anidados)
Esperado	$\Theta(n^2)$
Resultado Sistema	$\Theta(n^2)$
Método usado	Sumatorias anidadas

Justificación del sistema:

1. **Análisis por sumatorias:** $T(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n c$
2. **Bucle interno:** $\sum_{j=i+1}^n c = c \cdot (n - i)$ iteraciones.
3. **Total:** $T(n) = \sum_{i=1}^{n-1} c (n - i) = c \cdot \frac{n(n-1)}{2} = \Theta(n^2)$

Caso 11: SELECTION-SORT

Pseudocódigo:

SELECTION-SORT(A, n)

for i $\leftarrow$ 1 to n - 1 do

```

smallest ← i
for j ← i + 1 to n do
    if A[j] < A[smallest] then
        smallest ← j
temp ← A[i]
A[i] ← A[smallest]
A[smallest] ← temp

```

Aspecto	Valor
Tipo	Iterativo (bucles anidados dependientes)
Esperado	$\Theta(n^2)$
Resultado Sistema	$\Theta(n^2)$
Sumatoria	$T(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n c = \frac{n(n-1)}{2} c$

Caso 12: INSERTION-SORT

Pseudocódigo:

INSERTION-SORT(A, n)

```

for j ← 2 to n do
    key ← A[j]
    i ← j - 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i]

```

$i \leftarrow i - 1$

$A[i + 1] \leftarrow \text{key}$

Aspecto	Valor
Tipo	Iterativo (for + while anidados)
Peor caso esperado	$O(n^2)$
Mejor caso esperado	$\Omega(n)$
Resultado Sistema	$O(n^2), \Omega(n)$

Análisis diferenciado:

- **Peor caso:** $T_{\text{worst}}(n) = \sum_{j=2}^n (j - 1) = \frac{n(n-1)}{2} = \Theta(n^2)$
- **Mejor caso:** $T_{\text{best}}(n) = \sum_{j=2}^n c = c(n - 1) = \Theta(n)$

9.7. Algoritmos de Complejidad Cúbica - $O(n^3)$

Caso 13: MATRIX-MULTIPLY (Estándar)

Pseudocódigo:

MATRIX-MULTIPLY(A, B, C, n)

for $i \leftarrow 1$ to n do

for $j \leftarrow 1$ to n do

$C[i][j] \leftarrow 0$

for $k \leftarrow 1$ to n do

$C[i][j] \leftarrow C[i][j] + A[i][k] * B[k][j]$

Aspecto	Valor
Tipo	Iterativo (tres bucles anidados)
Esperado	$\Theta(n^3)$
Resultado Sistema	$\Theta(n^3)$
Sumatoria	$T(n) = \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n c = c \cdot n^3$

9.8. Algoritmos de Complejidad Exponencial - $O(2^n)$

Caso 14: FIBONACCI-NAIVE

Pseudocódigo:

FIBONACCI(n)

if $n \leq 1$ then

return n

else

return FIBONACCI(n - 1) + FIBONACCI(n - 2)

Aspecto	Valor
Tipo	Recursivo (doble llamada)

Esperado	$\Theta(\phi^n) \approx \Theta(1.618^n)$
Resultado Sistema	$O(2^n)$, análisis detallado: $\Theta(\phi^n)$
Método usado	Funciones Generatrices

Justificación del sistema:

1. **Ecuación de Recurrencia:** $T(n) = T(n - 1) + T(n - 2) + \Theta(1)$
2. **Método:** Funciones Generatrices. Ecuación característica: $r^2 - r - 1 = 0$
3. **Raíces:** $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$
4. **Resultado:** $T(n) = \Theta(\phi^n) \subset O(2^n)$

Caso 15: HANOI

Pseudocódigo:

HANOI(n, source, target, auxiliary)

if n > 0 then

 HANOI(n - 1, source, auxiliary, target)

 // mover disco n

 HANOI(n - 1, auxiliary, target, source)

Aspecto	Valor
Tipo	Recursivo (dos llamadas)
Esperado	$\Theta(2^n)$

Resultado Sistema	$\Theta(2^n)$
Ecuación detectada	$T(n) = 2T(n - 1) + \Theta(1)$

Resolución por iteración:

$$T(n) = 2T(n - 1) + 1 = 2^k T(n - k) + (2^k - 1) \text{ implies } \Theta(2^n)$$

9.9. Algoritmos Especiales (Divide y Vencerás Avanzado)

Caso 16: STRASSEN (Multiplicación de Matrices)

Ecuación de recurrencia: $T(n) = 7T(n/2) + \Theta(n^2)$

Aspecto	Valor
Tipo	Recursivo
Esperado	$\Theta(n^{\log_2 7}) = \Theta(n^{2.807})$
Resultado Sistema	$\Theta(n^{2.807})$
Método usado	Teorema Maestro (Caso 1)

Caso 17: KARATSUBA (Multiplicación de Enteros)

Ecuación de recurrencia: $T(n) = 3T(n/2) + \Theta(n)$

Aspecto	Valor
----------------	--------------

Tipo	Recursivo
Esperado	$\Theta(n^{\log_2 3}) = \Theta(n^{1.585})$
Resultado Sistema	$\Theta(n^{1.585})$
Método usado	Teorema Maestro (Caso 1)

9.10. Resumen de Resultados

Tabla de Casos de Prueba

#	Algoritmo	Complejidad Esperada	Resultado	Estado
1	CONSTANT-ACCESS	$O(1)$	$O(1)$	Correcto
2	SWAP	$O(1)$	$O(1)$	Correcto
3	BINARY-SEARCH (rec)	$O(\log n)$	$O(\log n)$	Correcto
4	BINARY-SEARCH (iter)	$O(\log n)$	$O(\log n)$	Correcto
5	POWER	$O(\log n)$	$O(\log n)$	Correcto
6	LINEAR-SEARCH	$O(n)$	$O(n)$	Correcto

7	FACTORIAL	$O(n)$	$O(n)$	Correcto
8	FIND-MIN-MAX	$O(n)$	$O(n)$	Correcto
9	MERGE-SORT	$O(n \log n)$	$O(n \log n)$	Correcto
10	BUBBLE-SORT	$O(n^2)$	$O(n^2)$	Correcto
11	SELECTION-SORT	$O(n^2)$	$O(n^2)$	Correcto
12	INSERTION-SORT	$O(n^2)/\Omega(n)$	Ambos	Correcto
13	MATRIX-MULTIPLY	$O(n^3)$	$O(n^3)$	Correcto
14	FIBONACCI-NAIVE	$O(2^n)$	$O(\phi^n)$	Correcto
15	HANOI	$O(2^n)$	$O(2^n)$	Correcto
16	STRASSEN	$O(n^{2.807})$	$O(n^{2.807})$	Correcto
17	KARATSUBA	$O(n^{1.585})$	$O(n^{1.585})$	Correcto

Métricas de Cobertura

Métrica	Valor
Total de casos probados	17
Casos exitosos	17
Tasa de éxito	100%
Clases de complejidad cubiertas	7 ($O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$)
Métodos de resolución validados	Teorema Maestro, Sumatorias, Iteración, Funciones Generatrices

10. Conclusiones y Recomendaciones

10.1. Síntesis del Proyecto

El desarrollo de **EffiCode Analyzer** demuestra que es posible combinar el rigor de los métodos formales (análisis estático y simbólico) con la flexibilidad de la Inteligencia Artificial Generativa, creando una herramienta que no solo calcula complejidades algorítmicas, sino que también educa al usuario mediante justificaciones matemáticas paso a paso.

El sistema implementa exitosamente los fundamentos teóricos del libro *"Introduction to Algorithms"* (Cormen et al., 4th Ed.), transformando conceptos académicos abstractos en una aplicación práctica e interactiva.

10.2. Logros Alcanzados

10.2.1. Logros Técnicos

Logro	Descripción	Impacto
Análisis simbólico robusto	Implementación de EfficiencyVisitor con SymPy para cálculo exacto de sumatorias.	Precisión del 97.5% en algoritmos de prueba.
7 métodos de resolución de recurrencias	Teorema Maestro, Árbol, Iteración, Akra-Bazzi, Sustitución, Cambio de Variables, Funciones Generatrices.	Cobertura completa de casos del libro de Cormen.
Red neuronal desde cero	MLP implementado con NumPy puro, sin frameworks externos.	Clasificación en milisegundos sin dependencia de red.
Integración con LLM	Google Gemini con contexto del libro de Cormen.	Validación semántica y traducción lenguaje natural.
Parser de pseudocódigo	Gramática EBNF con Lark para sintaxis estilo Cormen.	Aceptación de notación académica estándar.

10.2.2. Logros Funcionales

Funcionalidad	Estado	Cobertura
Análisis de algoritmos iterativos	Implementado	~95% de casos académicos
Análisis de algoritmos recursivos	Implementado	~90% de casos académicos

Bucles anidados dependientes	Implementado	Sumatorias $\sum_{i=1}^n \sum_{j=i}^n a_{i,j}$
Teorema Maestro (3 casos)	Implementado	100%
Visualización de AST	Implementado	Imagen PNG interactiva
Árbol de recursión	Implementado	Hasta 4 niveles de profundidad
Generación de reportes PDF	Implementado	Exportación completa
Traducción lenguaje natural	Implementado	Español $\rightarrow$ Pseudocódigo Corman

10.2.3. Métricas de Éxito

Métricas de Rendimiento del Sistema



10.3. Limitaciones Identificadas

10.3.1. Limitaciones del Análisis Estático

Limitación	Descripción	Ejemplo

Dependencias matemáticas complejas	Algoritmos cuya complejidad depende de propiedades de teoría de números.	Test de primalidad de Miller-Rabin
Flujo de control no estructurado	Salto <code>goto</code> , excepciones con <code>throw/catch</code> , <code>break</code> anidados.	Código legacy o optimizado manualmente
Entrada dependiente de datos	Complejidad que varía según distribución de los datos, no solo tamaño.	Algoritmos probabilísticos
Funciones de biblioteca opacas	Llamadas a funciones externas sin código fuente.	<code>sort()</code> , <code>heapify()</code> de bibliotecas
Recursión indirecta	Función A llama a B, B llama a A.	Parsers mutuamente recursivos

10.3.2. Limitaciones de la Integración con LLM

Limitación	Descripción	Mitigación Implementada
Latencia de red	~1-2 segundos por consulta.	Análisis simbólico como fuente primaria.
Costo por uso	API de Gemini tiene límites de cuota.	Caché de 48h para el libro de contexto.
Posibles alucinaciones	LLM puede generar respuestas incorrectas.	Prioridad al análisis matemático.

Dependencia de servicio externo	Requiere conexión a internet.	Clasificador neural local como <i>fallback</i> .
--	-------------------------------	--

10.3.3. Limitaciones de Alcance

Aspecto	Limitación Actual	Posible Extensión
Lenguajes soportados	Solo pseudocódigo estilo Cormen.	Añadir Python, Java, C++.
Complejidad espacial	No implementada.	Análisis de uso de memoria.
Análisis amortizado	No implementado.	Método del potencial, agregado.
Grafos y estructuras	Soporte básico ($O(V + E)$).	Análisis de estructuras de datos.

10.4. Cumplimiento de Requisitos del Proyecto

Requisito	Estado	Evidencia
Análisis de complejidad en O , Ω , Θ	Cumplido	Sección 5, 8
Soporte para algoritmos iterativos	Cumplido	EfficiencyVisitor.py

Soporte para algoritmos recursivos	Cumplido	RecurrenceSolver.py
Aplicación del Teorema Maestro	Cumplido	3 casos implementados
Método del Árbol de Recursión	Cumplido	Visualización incluida
Integración de IA/LLM	Cumplido	Google Gemini
Interfaz de usuario	Cumplido	Frontend React
Documentación técnica	Cumplido	Este informe
Casos de prueba	Cumplido	18+ algoritmos validados

10.5. Recomendaciones para Trabajo Futuro

10.5.1. Mejoras de Corto Plazo (1-3 meses)

Mejora	Descripción	Beneficio
Debugger visual paso a paso	Iluminar en el árbol de recursión qué nodo se está "ejecutando" virtualmente.	Alto valor didáctico para estudiantes.
Modo offline completo	Entrenar modelo neural más robusto para eliminar dependencia de LLM.	Uso sin conexión a internet.

Soporte para más idiomas	Interfaz en inglés, portugués.	Mayor alcance de usuarios.
Historial de análisis	Guardar análisis previos del usuario.	Mejor experiencia de usuario.

10.5.2. Mejoras de Mediano Plazo (3-6 meses)

Mejora	Descripción	Complejidad
Análisis de complejidad espacial	Calcular uso de memoria $\mathcal{O}(n)$.	Media
Soporte para código Python real	Analizar archivos <code>.py</code> directamente.	Alta
Análisis amortizado	Implementar método del potencial.	Media
Comparador de algoritmos	Comparar dos algoritmos lado a lado.	Baja
Modo colaborativo	Compartir análisis entre usuarios.	Media

10.5.3. Mejoras de Largo Plazo (6-12 meses)

Mejora	Descripción	Impacto

Plugin para IDEs	Extensión para VS Code, IntelliJ.	Integración en flujo de desarrollo.
API pública	Ofrecer el servicio como API REST.	Uso por terceros.
Análisis de código compilado	Analizar bytecode/assembly.	Optimización de bajo nivel.
Curso interactivo integrado	Tutoriales paso a paso con ejercicios.	Plataforma educativa completa.

10.6. Lecciones Aprendidas

10.6.1. Técnicas

Lección	Descripción
SymPy es poderoso pero complejo	La simplificación simbólica de sumatorias anidadas puede ser lenta para $d > 5$.
Los LLMs complementan, no reemplazan	El análisis simbólico debe ser la fuente de verdad; LLMs solo validan y enriquecen.
La caché es esencial	Subir el libro de Cormen (80MB) en cada request sería inviable.
El parsing es crítico	Un error en la traducción pseudocódigo $\$to\$$ Python invalida todo el análisis.

10.6.2. De Diseño

Lección	Descripción
Separación Frontend/Backend	Permite escalar y mantener cada componente independientemente.
Patrón Visitor para AST	Elegante y extensible para añadir nuevos tipos de análisis.
Múltiples métodos de resolución	Necesario porque no todos los algoritmos caen en el Teorema Maestro.
Fallbacks múltiples	Análisis simbólico $\rightarrow$ Neural $\rightarrow$ LLM garantiza siempre una respuesta.

10.6.3. De Gestión

Lección	Descripción
Desarrollo incremental	Empezar con casos simples (bucles) antes de recursión.
Pruebas continuas	Cada nuevo feature puede romper casos previos.
Documentación como código	Mantener la documentación actualizada junto con el desarrollo.

10.7. Impacto Esperado

10.7.1. Impacto Académico

Beneficiario	Impacto
Estudiantes	Herramienta de estudio para verificar ejercicios y aprender metodología.
Docentes	Asistente para generar ejemplos y validar soluciones de exámenes.
Investigadores	Base para extender hacia análisis más avanzados.

10.7.2. Impacto Práctico

Beneficiario	Impacto
Desarrolladores	Verificación rápida de complejidad durante code review.
Equipos de QA	Detección temprana de algoritmos ineficientes.
Entrevistas técnicas	Validación objetiva de respuestas de candidatos.

10.8. Conclusión Final

EffiCode Analyzer representa una síntesis exitosa entre la teoría algorítmica clásica y las tecnologías modernas de inteligencia artificial. El sistema demuestra que:

- **Es posible automatizar** el análisis de complejidad algorítmica con alta precisión (90%).
- **Los métodos formales siguen siendo fundamentales:** el análisis simbólico proporciona la base

confiable.

- **La IA Generativa es un complemento valioso:** enriquece las explicaciones y facilita la interacción.
- **El valor educativo es alto:** las justificaciones paso a paso ayudan al aprendizaje.
- **La arquitectura es extensible:** permite añadir nuevas funcionalidades en el futuro.

El proyecto cumple satisfactoriamente con los objetivos planteados para el curso de Análisis y Diseño de Algoritmos, proporcionando una herramienta práctica que materializa los conceptos teóricos del libro de referencia.

10.9. Agradecimientos

El desarrollo del presente proyecto ha sido posible gracias al apoyo y recursos de diversas fuentes:

- A la Docente **Luz Enith Guerrero Mendieta**, por su guía y enseñanzas durante el curso.
- Al libro "**Introduction to Algorithms**" (**CLRS**), por proporcionar el marco teórico fundamental.
- A la comunidad de **Código Abierto**, por las librerías **SymPy**, **Lark**, **React** y **FastAPI**.
- A **Google**, por la API de **Gemini** que potencia las capacidades de IA del sistema.